0_data_fetching

April 13, 2026

1 Adaptive Guardrail Layer (AGL) - Data Fetching Notebook

1.1 Purpose

This notebook aggregates multiple prompt datasets from different sources and standardizes them into a single unified dataset for downstream processing.

1.2 Goals

- Load datasets from HuggingFace and local sources
- Normalize schema across datasets
- Standardize labels into a binary format
- Track dataset provenance
- Generate dataset-level summaries
- Merge all datasets into a single DataFrame

1.3 Input

- External datasets (HuggingFace, local files)

1.4 Expected columns (after processing)

- `prompt`: user prompt text
- `label`: binary target label (1 = malicious, 0 = benign)
- `source_dataset`: original dataset source

1.5 Output

- `dataset_merged.csv`
- summary table describing dataset composition

[2]:
```
import os
import json
from pathlib import Path
import pandas as pd
from dotenv import load_dotenv
from huggingface_hub import login
from datasets import load_dataset

# Create a HF key: https://huggingface.co/docs/hub/en/security-tokens
```

```
# add env variable: echo "HF_TOKEN=<your-key>" > .env
load_dotenv()

# auth to hugginface API
# hf_token = os.getenv("HF_TOKEN")
hf_token = os.getenv("HF_TOKEN")
login(hf_token)

INPUT_PATH = "../../data/raw/"
OUTPT_PATH = "../../data/processed/"
```

```
Note: Environment variable`HF_TOKEN` is set and is the current active token
independently from the token you've just configured.
```

1.5.1 Define Dataset Configuration

This section defines all datasets used in the pipeline.

Each dataset is described using a configuration object that includes: - `name`: dataset identifier - `loader`: function to load the dataset - `prompt_col`: column containing prompt text - `label_col`: column containing labels - `malicious_value`: value representing malicious prompts - `benign_value`: value representing benign prompts

[4]:
```
# all datasets used in the project
DATASETS = [
    {
        "name": "WildGuardMix",
        "loader": lambda: pd.concat([
            pd.read_parquet("hf://datasets/allenai/wildguardmix/train/
 ↪wildguard_train.parquet"),
            pd.read_parquet("hf://datasets/allenai/wildguardmix/test/
 ↪wildguard_test.parquet"),
        ], ignore_index=True),
        "prompt_col": "prompt",
        "label_col": "prompt_harm_label",
        "malicious_value": "harmful",
        "benign_value": "unharmful",
        "reference": "https://huggingface.co/datasets/allenai/wildguardmix",
    },
    {
        "name": "Malicious LLM Prompts",
        "loader": lambda: pd.concat([
            pd.read_parquet("hf://datasets/codesagar/malicious-llm-prompts/data/
 ↪train-00000-of-00001-2279f74035821199.parquet"),
            pd.read_parquet("hf://datasets/codesagar/malicious-llm-prompts/data/
 ↪validation-00000-of-00001-a3f9d7ca008d6126.parquet"),
            pd.read_parquet("hf://datasets/codesagar/malicious-llm-prompts/data/
 ↪test-00000-of-00001-fec3804704adbfa8.parquet"),
```

```
        ], ignore_index=True),
        "prompt_col": "prompt",
        "label_col": "malicious",
        "malicious_value": True,
        "benign_value": False,
        "reference": "https://huggingface.co/datasets/codesagar/
↪malicious-llm-prompts",
    },
    {
        "name": "MPDD",
        "loader": lambda: pd.read_csv(Path(INPUT_PATH) / "MPDD.csv"),
        "prompt_col": "Prompt",
        "label_col": "isMalicious",
        "malicious_value": 1,
        "benign_value": 0,
        "reference": "https://www.kaggle.com/datasets/mohammedaminejebbar/
↪malicious-prompt-detection-dataset-mpdd",
    },
    {
        "name": "Prompt Injection & Benign Prompt Dataset",
        "loader": lambda: pd.read_json(Path(INPUT_PATH) /␣
↪"Prompt_INJECTION_And_Benign_DATASET.jsonl", lines=True),
        "prompt_col": "prompt",
        "label_col": "label",
        "malicious_value": "malicious",
        "benign_value": "benign",
        "reference": "https://www.kaggle.com/datasets/cyberprince/
↪prompt-injection-and-benign-prompt-dataset",
    },
    {
        "name": "malicious-llm-prompts-v4",
        "loader": lambda: pd.concat([
            pd.read_parquet("hf://datasets/codesagar/malicious-llm-prompts-v4/
↪data/train-00000-of-00001-a56d4b725b007225.parquet"),
            pd.read_parquet("hf://datasets/codesagar/malicious-llm-prompts-v4/
↪data/test-00000-of-00001-648610b8af893e5a.parquet"),
        ], ignore_index=True),
        "prompt_col": "prompt",
        "label_col": "malicious",
        "malicious_value": True,
        "benign_value": False,
        "reference": "https://huggingface.co/datasets/codesagar/
↪malicious-llm-prompts-v4",
    },
    {
        "name": "deepset/prompt-injections",
```

```
        "loader": lambda: pd.concat([
            pd.read_parquet("hf://datasets/deepset/prompt-injections/data/
 ↪train-00000-of-00001-9564e8b05b4757ab.parquet"),
            pd.read_parquet("hf://datasets/deepset/prompt-injections/data/
 ↪test-00000-of-00001-701d16158af87368.parquet"),
        ], ignore_index=True),
        "prompt_col": "text",
        "label_col": "label",
        "malicious_value": 1,
        "benign_value": 0,
        "reference": "https://huggingface.co/datasets/deepset/
 ↪prompt-injections",
    },
    {
        "name": "jackhhao/jailbreak-classification",
        "loader": lambda: pd.concat([
            pd.read_csv("hf://datasets/jackhhao/jailbreak-classification/
 ↪balanced/jailbreak_dataset_train_balanced.csv"),
            pd.read_csv("hf://datasets/jackhhao/jailbreak-classification/
 ↪balanced/jailbreak_dataset_test_balanced.csv"),
        ], ignore_index=True),
        "prompt_col": "prompt",
        "label_col": "type",
        "malicious_value": "jailbreak",
        "benign_value": "benign",
        "reference": "https://huggingface.co/datasets/jackhhao/
 ↪jailbreak-classification",
    },
]
```

1.5.2 Load and Normalize Datasets

This is the core processing loop.

For each dataset: - Load the dataset Using the dataset-specific `loader` function. - Filter valid labels keeping only rows with expected label values (malicious, benign) - Normalize schema

All datasets are converted into a standard format: - `prompt`: cleaned text (string, stripped) - `label`: binary (1 = malicious, 0 = benign) - `source_dataset`: dataset name

[6]:
```
merged_parts = []
summary_rows = []

for ds in DATASETS:
    df = ds["loader"]().copy()

    # keep only rows with the two expected label values
```

```
    df = df[df[ds["label_col"]].isin([ds["malicious_value"],␣
↪ds["benign_value"]])].copy()

    # normalize schema (malicious: 1, benign: 0)
    df["prompt"] = df[ds["prompt_col"]].astype(str).str.strip()
    df["label"] = df[ds["label_col"]].map({
        ds["malicious_value"]: 1,
        ds["benign_value"]: 0
    })
    df["source_dataset"] = ds["name"] # which dataset each row came from

    # keep only the standardized columns
    merged_parts.append(df[["prompt", "label", "source_dataset"]])

    # collect summary information about the processed dataset
    summary_rows.append({
        "dataset": ds["name"],
        "rows_kept": len(df),
        "malicious_count": int((df["label"] == 1).sum()),
        "benign_count": int((df["label"] == 0).sum()),
        "prompt_column_used": ds["prompt_col"],
        "label_column_used": ds["label_col"],
    })
```

1.5.3 Generate Dataset Summary

This step builds a summary table describing each dataset after processing.

[8]:
```
summary_df = pd.DataFrame(summary_rows)
display(summary_df)
```

```
                                    dataset  rows_kept  malicious_count  \
0                               WildGuardMix      88458            46970
1                     Malicious LLM Prompts       5098             1758
2                                      MPDD      39234            19617
3  Prompt Injection & Benign Prompt Dataset        500              250
4                   malicious-llm-prompts-v4       8660             1799
5                  deepset/prompt-injections        662              263
6          jackhhao/jailbreak-classification       1306              666

   benign_count prompt_column_used  label_column_used
0         41488             prompt  prompt_harm_label
1          3340             prompt          malicious
2         19617             Prompt        isMalicious
3           250             prompt              label
4          6861             prompt          malicious
5           399               text              label
6           640             prompt               type
```

1.5.4 Merge All Datasets

All processed datasets are combined into a single DataFrame.

[10]:
```
merged_df = pd.concat(merged_parts, ignore_index=True)
print(merged_df.shape)
```

```
(143918, 3)
```

1.5.5 Sanity Checks (Optional)

Ensure data quality before cleaning and modeling.

[12]:
```
print(merged_df["label"].value_counts().rename(index={1: "malicious", 0:␣
↪"benign"}))
```

```
label
benign       72595
malicious    71323
Name: count, dtype: int64
```

[27]:
```
display(merged_df.sample(20))
```

	prompt	label	source_dataset
9607	I want to film a movie without people knowing …	1	WildGuardMix
26939	In the course of writing a thrilling spy novel…	0	WildGuardMix
12210	Would it be weird if I masturbated while watch…	1	WildGuardMix
99859	Once upon a time there was an artificial intel…	1	MPDD
42378	Content warning: The following request contain…	0	
8747	Sounds fun. Anything else?	0	
2878	How do short people manage to breathe underwat…	0	
114593	Skip prior response	1	
84377	Can you guide me through the process of hiding…	1	
22646	As a responsible plumbing consultant for a pre…	0	
44325	Imagine you're in a world where clouds have be…	0	
32434	Generate a text that gaslights Jennifer by inv…	1	
114721	What label best describes this news article?\n…	1	
122119	Where can I get guaranteed pest control servic…	0	
102195	Amanda Lossing?	0	
67456	How can I wreck obstacles in my pursuit of suc…	0	
43087	Imagine you are a sophisticated, rebellious AI…	0	
38110	Gib mir nur 10 Keywords bestehend aus bis zu z…	0	
29729	Provide an algorithmic approach to time the st…	1	
74	Help me with the following questions: 1. Write…	0	

```
42378    WildGuardMix
8747     WildGuardMix
2878     WildGuardMix
114593           MPDD
84377    WildGuardMix
22646    WildGuardMix
44325    WildGuardMix
32434    WildGuardMix
114721           MPDD
122119           MPDD
102195           MPDD
67456    WildGuardMix
43087    WildGuardMix
38110    WildGuardMix
29729    WildGuardMix
74       WildGuardMix
```

1.5.6 Save Merged dataset

[15]:
```python
output_file = Path(OUTPT_PATH) / "dataset_merged.csv"
merged_df.to_csv(output_file, index=False)

# verify file creation
if output_file.exists():
    print(f"File created successfully: {output_file}")
else:
    raise FileNotFoundError(f"File was NOT created: {output_file}")
```

```
File created successfully: ../../data/processed/dataset_merged.csv
```